LAPORAN PRAKTIKUM
STRUKTUR DATA

Modul 7
STACK



Disusun Oleh :
Dina Aulia Rizky (101902500011)

Dosen:
Diah Septiani, S.Kom., M.C.S.(AI)

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2026

STACK.h

```
#ifndef STACK_H
#define STACK_H

const int MAKS = 20;

struct Stack {
    int info[MAKS + 1];   // indeks data 1..20
    int Top;              // 0 = kosong, 20 = penuh
};

void createStack(Stack &S);
bool isEmpty(Stack S);
bool isFull(Stack S);
bool push(Stack &S, int x);
bool pop(Stack &S, int &x);
void printInfo(Stack S);
void balikStack(Stack &S);

#endif
```

Stack.cpp

```
#include <iostream>
#include "stack.h"
using namespace std;

void createStack(Stack &S) {
    S.Top = 0;
}

bool isEmpty(Stack S) {
    return S.Top == 0;
}

bool isFull(Stack S) {
    return S.Top == MAKS;
```

```
}

bool push(Stack &S, int x) {
    if (isFull(S)) return false;   // tidak mengubah isi maupun
Top
    S.Top++;
    S.info[S.Top] = x;
    return true;
}

bool pop(Stack &S, int &x) {
    if (isEmpty(S)) return false;  // tidak mengubah isi maupun
Top
    x = S.info[S.Top];
    S.Top--;
    return true;
}

void printInfo(Stack S) {
    if (isEmpty(S)) {
        cout << "Stack kosong." << endl;
        return;
    }
    cout << "TOP: ";
    for (int i = S.Top; i >= 1; i--) {
        cout << S.info[i] << " ";
    }
    cout << endl;
}

void balikStack(Stack &S) {
    Stack bantu;
    createStack(bantu);
    int x, y, n = 0;

    while (pop(S, x)) {
        push(bantu, x);
        n++;
    }
```

```
    for (int sisa = n; sisa >= 1; sisa--) {
        for (int i = 0; i < sisa - 1; i++) {   // parkir elemen
di atas dasar ke S
            pop(bantu, y);
            push(S, y);
        }
        pop(bantu, x);                          // x = elemen
dasar bantu
        for (int i = 0; i < sisa - 1; i++) {   // kembalikan
parkiran ke bantu
            pop(S, y);
            push(bantu, y);
        }
        push(S, x);                             // x menempati
posisi berikutnya di S
    }
}
```

main.cpp

```
#include <iostream>
#include "stack.h"
using namespace std;

int main() {
    Stack S;
    createStack(S);
    int pilih, x;

    do {
        cout << "\n=== MENU STACK (ARRAY) ===" << endl;
        cout << "1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Tampilkan" << endl;
        cout << "4. Balik Stack" << endl;
        cout << "0. Keluar" << endl;
```

```
        cout << "Pilihan: ";
        if (!(cin >> pilih)) break;

        switch (pilih) {
        case 1:
            cout << "Masukkan bilangan: ";
            cin >> x;
            if (push(S, x))
                cout << x << " berhasil di-Push." << endl;
            else
                cout << "Push ditolak: stack penuh (maks. 20
elemen)." << endl;
            break;
        case 2:
            if (pop(S, x))
                cout << "Nilai yang di-Pop: " << x << endl;
            else
                cout << "Pop ditolak: stack kosong." << endl;
            break;
        case 3:
            printInfo(S);
            break;
        case 4:
            balikStack(S);
            cout << "Stack berhasil dibalik." << endl;
            printInfo(S);
            break;
        case 0:
            cout << "Program selesai." << endl;
            break;
        default:
            cout << "Pilihan tidak valid." << endl;
        }
    } while (pilih != 0);

    return 0;
}
```

Output

```
PS D:\Modul 7\Soal_1> g++ main.cpp stack.cpp -o soal1.exe
PS D:\Modul 7\Soal_1> .\soal1.exe

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 1
Masukkan bilangan: 7
7 berhasil di-Push.

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 1
Masukkan bilangan: 12
12 berhasil di-Push.

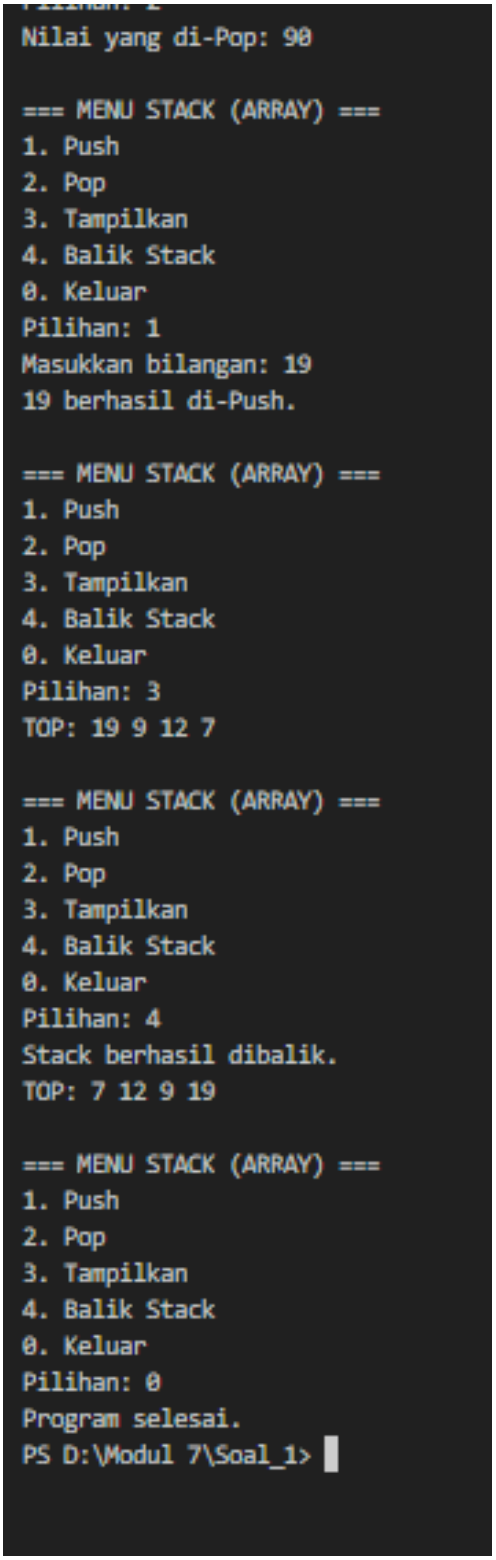
=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 1
Masukkan bilangan: 5
5 berhasil di-Push.

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 2
Nilai yang di-Pop: 5

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik 1
Masukkan bilangan: 9
9 berhasil di-Push.

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 1
Masukkan bilangan: 90
90 berhasil di-Push.

=== MENU STACK (ARRAY) ===
1. Push
2. Pop
3. Tampilkan
4. Balik Stack
0. Keluar
Pilihan: 2
```



Kesimpulan

Berdasarkan praktikum yang telah dilakukan, ADT Stack berhasil diimplementasikan dengan representasi array pada file stack.h, stack.cpp, dan

main.cpp. Stack menyimpan bilangan bulat pada array info[21] dengan kapasitas 20 elemen (indeks 1-20) dan satu field Top yang menunjuk elemen teratas. Top bernilai 0 saat stack kosong dan 20 saat penuh, yang diperiksa melalui fungsi isEmpty dan isFull. Operasi push dan pop mengembalikan nilai false jika gagal, sehingga isi stack dan Top tidak berubah saat Push pada stack penuh atau Pop pada stack kosong. Pada program ini, data ditampilkan dari Top ke bawah, sehingga data yang terakhir dimasukkan muncul lebih dulu dan juga yang pertama keluar saat Pop, sesuai prinsip LIFO. Fungsi balikStack berhasil membalik isi stack, misalnya dari TOP: 19 9 12 7 menjadi TOP: 7 12 9 19, dengan jumlah elemen yang tetap dan hanya memakai satu stack bantu melalui operasi Push dan Pop.

SOAL 2

STACK.h

```
#ifndef STACK_H
#define STACK_H

struct Node {
    int info;
    Node *next;
};
typedef Node *Address;

struct Stack {
    Address Top;          // kosong jika Top == nullptr
};

void createStack(Stack &S);
bool isEmpty(Stack S);
Address alokasi(int x);
void dealokasi(Address &P);
void push(Stack &S, int x);
bool pop(Stack &S, int &x);
void printInfo(Stack S);
void pushAscending(Stack &S, int x);
```

```
void clearStack(Stack &S);
#endif
```

Stack.cpp

```
#include <iostream>
#include "stack.h"
using namespace std;

void createStack(Stack &S) {
    S.Top = nullptr;
}

bool isEmpty(Stack S) {
    return S.Top == nullptr;
}

Address alokasi(int x) {
    Address P = new Node;
    P->info = x;
    P->next = nullptr;
    return P;
}

void dealokasi(Address &P) {
    delete P;
    P = nullptr;
}

// Menambah elemen pada TOP
void push(Stack &S, int x) {
    Address P = alokasi(x);
    P->next = S.Top;
    S.Top = P;
}

// Mengambil nilai TOP dan melepas node tersebut
bool pop(Stack &S, int &x) {
```

```
    if (isEmpty(S)) return false;   // keadaan stack tidak
berubah
    Address P = S.Top;
    x = P->info;
    S.Top = P->next;
    dealokasi(P);
    return true;
}

void printInfo(Stack S) {
    if (isEmpty(S)) {
        cout << "Stack kosong." << endl;
        return;
    }
    cout << "TOP: ";
    for (Address P = S.Top; P != nullptr; P = P->next) {
        cout << P->info << " ";
    }
    cout << endl;
}
void pushAscending(Stack &S, int x) {
    Stack bantu;
    createStack(bantu);
    int y;

    while (!isEmpty(S) && S.Top->info > x) {
        pop(S, y);
        push(bantu, y);
    }
    push(S, x);
    while (pop(bantu, y)) {
        push(S, y);
    }
    // stack bantu kosong kembali (Top = nullptr)
}

// Melepas seluruh node dan mengembalikan Top menjadi nullptr
void clearStack(Stack &S) {
    int x;
    while (pop(S, x)) {
```

```
    }
    S.Top = nullptr;
}
```

main.cpp

```
#include <iostream>
#include "stack.h"
using namespace std;

int main() {
    Stack S;
    createStack(S);
    int pilih, x;

    do {
        cout << "\n=== MENU STACK (POINTER, TERURUT) ===" <<
endl;
        cout << "1. Tambah Terurut" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Tampilkan" << endl;
        cout << "0. Keluar" << endl;
        cout << "Pilihan: ";
        if (!(cin >> pilih)) break;

        switch (pilih) {
        case 1:
            cout << "Masukkan bilangan: ";
            cin >> x;
            pushAscending(S, x);
            cout << x << " berhasil ditambahkan secara
terurut." << endl;
            break;
        case 2:
            if (pop(S, x))
                cout << "Nilai yang di-Pop: " << x << endl;
            else
                cout << "Pop ditolak: stack kosong." << endl;
```

```
            break;
        case 3:
            printInfo(S);
            break;
        case 0:
            cout << "Program selesai." << endl;
            break;
        default:
            cout << "Pilihan tidak valid." << endl;
        }
    } while (pilih != 0);

    clearStack(S);   // lepas semua node sebelum program
berakhir
    return 0;
}
```

Output

```
PS D:\Modul 7> cd Soal_2
PS D:\Modul 7\Soal_2> g++ main.cpp stack.cpp -o soal2.exe
PS D:\Modul 7\Soal_2> .\soal2.exe

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 1
Masukkan bilangan: 3
3 berhasil ditambahkan secara terurut.

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 1
Masukkan bilangan: 7
7 berhasil ditambahkan secara terurut.

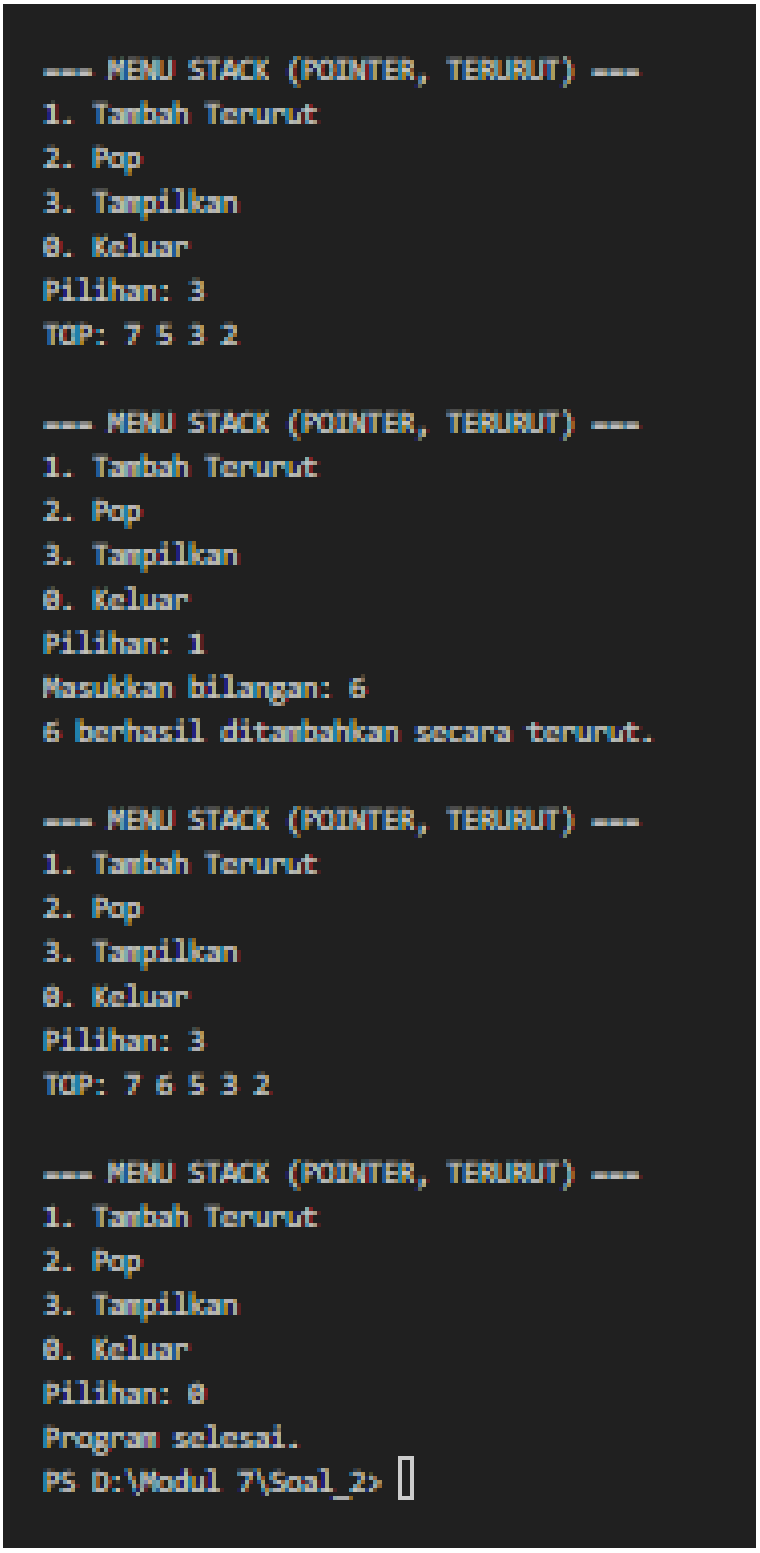
=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 1
Masukkan bilangan: 2
2 berhasil ditambahkan secara terurut.

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 1
Masukkan bilangan: 7
7 berhasil ditambahkan secara terurut.

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 1
Masukkan bilangan: 5
5 berhasil ditambahkan secara terurut.

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 3
TOP: 7 7 5 3 2

=== MENU STACK (POINTER, TERURUT) ===
1. Tambah Terurut
2. Pop
3. Tampilkan
0. Keluar
Pilihan: 2
Nilai yang di-Pop: 7
```



Kesimpulan

Berdasarkan praktikum yang telah dilakukan, ADT Stack berhasil diimplementasikan dengan representasi pointer berbasis singly linked list pada

file stack.h, stack.cpp, dan main.cpp. Setiap elemen (Node) menyimpan data bertipe int pada field info dan satu pointer next yang menunjuk ke node di bawahnya, sedangkan stack hanya menyimpan pointer Top dan dinyatakan kosong jika Top bernilai nullptr. Alokasi memori memakai new dan dealokasi memakai delete, dan clearStack melepas seluruh node sebelum program berakhir. Penambahan data selalu melalui pushAscending, yang memindahkan elemen lebih besar ke stack bantu, menyisipkan nilai baru, lalu mengembalikan elemen tersebut, sehingga urutan menaik dari bawah ke Top selalu terjaga. Pada program ini, data ditampilkan dari Top ke bawah, sehingga nilai terbesar muncul lebih dulu, misalnya TOP: 7 7 5 3 2 setelah memasukkan 3, 7, 2, 7, 5. Nilai duplikat tetap disimpan sebagai elemen terpisah, dan Pop pada stack kosong ditolak tanpa mengubah keadaan stack.